PYTHON PROGRAMMING

Tasks 11–20

Problem Statements • Algorithms • Flowcharts • Sample Input/Output • Mermaid Flowchart Code

Prepared as a beginner-friendly practical record reference. The flowcharts are provided as readable step sequences and Mermaid code.

Task 11. Examination Result and Grace-Mark Validation

Problem Statement

Write a Python program to read marks for multiple subjects, validate that each mark is between 0 and 100, apply a maximum of 5 grace marks when a subject score is from 30 to 34, and calculate the final result and classification. Grace marks must not make a subject score exceed 100.

Algorithm

1. Start the program.

2. Read the number of subjects. Set total marks and failed-subject count to zero.

3. For each subject, read the marks and check that the value is between 0 and 100.

4. If marks are 35 or above, count the subject as passed and add the marks to the total.

5. If marks are from 30 to 34, add only enough grace marks to reach 35, then count the subject as passed.

6. Otherwise, count the subject as failed and add the original marks to the total.

7. After all subjects, display total marks and failed-subject count.

8. If no subject failed, calculate percentage and display classification: Distinction (75%+), First Class (60%+), Second Class (50%+), otherwise Pass. If any subject failed, display Fail.

9. Stop.

Flowchart (Text Form)

Start → Read subject count → For each subject: read and validate marks → Check pass mark / grace eligibility → Update total and failure count → More subjects? → Display result → If all passed, calculate percentage and classification → Stop.

Sample Input

```
Number of subjects: 3
Subject 1 marks: 80
Subject 2 marks: 32
Subject 3 marks: 70
```

Sample Output

```
Subject 1: Pass
Subject 2: Pass with 3 grace marks (final 35)
Subject 3: Pass
Total Marks: 185 / 300
Percentage: 61.67%
Failed Subjects: 0
Classification: First Class
Final Result: PASS
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Read number of subjects/]
    B --> C[Set total = 0, failed = 0]
    C --> D[/Read subject marks/]
    D --> E{Marks between 0 and 100?}
    E -- No --> F[Report invalid marks]
    E -- Yes --> G{Marks >= 35?}
    G -- Yes --> H[Add marks to total]
    G -- No --> I{Marks >= 30?}
    I -- Yes --> J[Add grace to reach 35; update total]
    I -- No --> K[Increase failed; add original marks]
    F --> L{More subjects?}
    H --> L
    J --> L
    K --> L
    L -- Yes --> D
    L -- No --> M{failed == 0?}
    M -- Yes --> N[Calculate percentage and classification]
    M -- No --> O[Display FAIL]
    N --> P([Stop])
    O --> P
```

Task 12. Attendance Recovery Computation

Problem Statement

Write a Python program to calculate a student's current attendance percentage and determine the minimum number of consecutive future classes the student must attend to reach a required attendance threshold. Assume the student attends every future class.

Algorithm

1. Start the program.

2. Read classes attended, total classes conducted, and required attendance percentage.

3. Validate that total classes is positive, attended classes are not negative or greater than total, and threshold is between 0 and 100.

4. Calculate current attendance percentage = attended / total × 100.

5. If current percentage meets the threshold, display zero additional classes.

6. Otherwise, start additional classes at zero and increase it one at a time until (attended + additional) / (total + additional) × 100 reaches the threshold.

7. Display the minimum additional classes and the resulting attendance percentage.

8. Stop.

Flowchart (Text Form)

Start → Read attendance details → Validate input → Calculate current percentage → Threshold met? → Yes: additional classes = 0 → No: increment future attended classes until threshold is reached → Display result → Stop.

Sample Input

```
Classes attended: 70
Total classes: 100
Required attendance: 75%
```

Sample Output

```
Current Attendance: 70.00%
Attendance Not Eligible
Minimum Consecutive Classes Required: 20
Attendance After Attending Them: 75.00%
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Read attended, total, threshold/]
    B --> C{Inputs valid?}
    C -- No --> D[Display invalid input]
    C -- Yes --> E[Calculate current percentage]
    E --> F{Percentage >= threshold?}
    F -- Yes --> G[Additional classes = 0]
    F -- No --> H[Set additional = 0]
    H --> I{Future percentage >= threshold?}
    I -- No --> J[additional = additional + 1]
    J --> I
    I -- Yes --> K[Display minimum classes and final percentage]
    G --> K
    D --> L([Stop])
    K --> L
```

Task 13. ATM Transaction Sequence Simulation

Problem Statement

Write a Python program to simulate a sequence of ATM deposits and withdrawals. Maintain the account balance, reject invalid transaction amounts, enforce a maximum withdrawal of ■10,000 per transaction, prevent withdrawals greater than the available balance, and display a transaction summary.

Algorithm

1. Start the program and read the initial balance and number of transactions.

2. Set total deposits and total withdrawals to zero.

3. Repeat for the given number of transactions: read transaction type and amount.

4. If the amount is not positive, reject the transaction.

5. If the type is Deposit, add the amount to balance and total deposits.

6. If the type is Withdrawal, first check the ■10,000 per-transaction limit, then check available balance. If both checks pass, subtract the amount and update total withdrawals.

7. If the transaction type is neither Deposit nor Withdrawal, report an invalid transaction.

8. After all transactions, display final balance, total deposits, and total withdrawals.

9. Stop.

Flowchart (Text Form)

Start → Read opening balance and transaction count → Read transaction type and amount → Validate amount → Deposit or withdrawal? → For withdrawal check limit and balance → Update balance and totals → More transactions? → Print summary → Stop.

Sample Input

```
Initial balance: 20000
Number of transactions: 3
Transaction 1: Deposit, amount 5000
Transaction 2: Withdraw, amount 3000
Transaction 3: Withdraw, amount 15000
```

Sample Output

```
Transaction 1: Deposit successful
Transaction 2: Withdrawal successful
Transaction 3: Rejected - withdrawal limit exceeded
Final Balance: ■22000
Total Deposits: ■5000
Total Withdrawals: ■3000
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Read opening balance and transaction count/]
    B --> C[Set deposit total = 0; withdrawal total = 0]
    C --> D[/Read type and amount/]
    D --> E{Amount > 0?}
    E -- No --> F[Reject invalid amount]
    E -- Yes --> G{Transaction is Deposit?}
    G -- Yes --> H[Add amount to balance and deposit total]
    G -- No --> I{Transaction is Withdrawal?}
    I -- No --> J[Reject invalid transaction type]
    I -- Yes --> K{Amount <= 10000?}
    K -- No --> L[Reject: limit exceeded]
    K -- Yes --> M{Amount <= balance?}
    M -- No --> N[Reject: insufficient balance]
    M -- Yes --> O[Subtract amount; update withdrawal total]
    F --> P{More transactions?}
    H --> P
    J --> P
    L --> P
    N --> P
    O --> P
    P -- Yes --> D
    P -- No --> Q[Display balance and transaction totals]
    Q --> R([Stop])
```

Task 14. Warehouse Inventory Reconciliation

Problem Statement

Write a Python program to reconcile inventory for multiple products. For each product, read opening stock, incoming quantity, dispatched quantity, and actual closing stock. Calculate expected closing stock, identify any discrepancy, and calculate the quantity required to restore stock to a minimum level of 10 units.

Algorithm

1. Start the program and read the number of products.

2. For each product, read its name, opening stock, incoming quantity, dispatched quantity, and actual closing stock.

3. Validate that all quantities are non-negative.

4. Calculate expected closing stock = opening stock + incoming quantity – dispatched quantity.

5. Compare actual closing stock with expected closing stock. If equal, report reconciled; otherwise, display the difference (actual – expected).

6. If actual closing stock is below 10, calculate replenishment = 10 – actual closing stock. Otherwise, report sufficient stock.

7. Repeat for all products and display completion message.

8. Stop.

Flowchart (Text Form)

Start → Read product count → Read stock details for each product → Validate quantities → Calculate expected closing stock → Compare with actual stock → Calculate replenishment if below 10 → More products? → Stop.

Sample Input

```
Number of products: 2
Product 1: Pen
Opening: 50, Incoming: 30, Dispatched: 20, Actual closing: 60
Product 2: Notebook
Opening: 20, Incoming: 10, Dispatched: 25, Actual closing: 3
```

Sample Output

```
Pen: Expected closing = 60; Actual closing = 60; Stock Reconciled; Replenishment = 0
Notebook: Expected closing = 5; Actual closing = 3; Discrepancy = -2; Replenishment required = 7
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Read number of products/]
    B --> C[/Read product stock details/]
    C --> D{All quantities >= 0?}
    D -- No --> E[Report invalid quantities]
    D -- Yes --> F[Expected = opening + incoming - dispatched]
    F --> G{Actual closing == expected?}
    G -- Yes --> H[Report stock reconciled]
    G -- No --> I[Display discrepancy]
    H --> J{Actual closing < 10?}
    I --> J
    J -- Yes --> K[Replenishment = 10 - actual closing]
    J -- No --> L[Report sufficient stock]
    E --> M{More products?}
    K --> M
    L --> M
    M -- Yes --> C
    M -- No --> N([Stop])
```

Task 15. Smart Parking Fee Management

Problem Statement

Write a Python program to calculate parking charges using vehicle type and entry/exit hours. Use ■30 per hour for a car and ■15 per hour for a bike, charge at least one hour, and cap the normal daily charge at ■200. Validate the time and vehicle type. This simplified model uses whole-hour times within one 24-hour period.

Algorithm

1. Start the program.

2. Read entry hour, exit hour, and vehicle type.

3. Validate that both hours are from 0 to 23 and vehicle type is car or bike.

4. Calculate duration as exit hour – entry hour. If exit is earlier than entry, treat it as crossing midnight and calculate (24 – entry) + exit.

5. If duration is zero, charge for one hour.

6. Set hourly rate to ■30 for car or ■15 for bike.

7. Calculate fee = duration × hourly rate, then cap the fee at ■200.

8. Display duration, vehicle type, and parking fee.

9. Stop.

Flowchart (Text Form)

Start → Read entry/exit hours and vehicle type → Validate → Calculate duration (including midnight crossing) → Apply minimum one hour → Select hourly rate → Calculate and cap fee → Display charge → Stop.

Sample Input

```
Entry hour: 9
Exit hour: 14
Vehicle type: car
```

Sample Output

```
Parking Duration: 5 hours
Vehicle Type: car
Parking Fee: ■150
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Read entry hour, exit hour, vehicle type/]
    B --> C{Time and vehicle valid?}
    C -- No --> D[Display invalid input]
    C -- Yes --> E{Exit hour < entry hour?}
    E -- Yes --> F[Duration = 24 - entry + exit]
    E -- No --> G[Duration = exit - entry]
    F --> H{Duration == 0?}
    G --> H
    H -- Yes --> I[Set duration = 1]
    H -- No --> J[Select vehicle hourly rate]
    I --> J
    J --> K[Fee = duration x rate; cap at 200]
    K --> L[Display parking fee]
    D --> M([Stop])
    L --> M
```

Task 16. Multi-Day Water Tank Simulation

Problem Statement

Write a Python program to simulate a water tank over multiple days. Update the water level using daily inflow and outflow, prevent the displayed level from going below zero or above tank capacity, report shortages and overflow, and identify the first day the water level reaches a specified threshold.

Algorithm

1. Start the program and read tank capacity, initial water level, number of days, and target threshold.
2. Validate that capacity is positive and the initial level and threshold are within the tank capacity.
3. For each day, read inflow and outflow. Reject negative quantities.
4. Calculate new level = current level + inflow – outflow.
5. If the new level is greater than capacity, report overflow and set level to capacity.
6. If the new level is below zero, report shortage and set level to zero.
7. Display the water level for that day.
8. If the level is at least the target threshold and this is the first such day, report the day reached.
9. Continue through all days and display final water level.
10. Stop.

Flowchart (Text Form)

Start → Read tank details → Validate → For each day read inflow/outflow → Calculate new level → Check overflow → Check shortage → Display level → Check threshold reached first time → More days? → Display final level → Stop.

Sample Input

```
Tank capacity: 1000
Initial water level: 300
Number of days: 3
Target threshold: 700
Day 1 inflow: 200, outflow: 100
Day 2 inflow: 300, outflow: 50
Day 3 inflow: 200, outflow: 50
```

Sample Output

```
Day 1 water level: 400 litres
Day 2 water level: 650 litres
Day 3 water level: 800 litres
Target threshold reached first on Day 3
Final water level: 800 litres
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Read capacity, initial level, days, threshold/]
    B --> C{Initial values valid?}
    C -- No --> D[Display invalid input]
    C -- Yes --> E[/Read daily inflow and outflow/]
    E --> F[New level = old level + inflow - outflow]
    F --> G{New level > capacity?}
    G -- Yes --> H[Report overflow; set level to capacity]
    G -- No --> I{New level < 0?}
    I -- Yes --> J[Report shortage; set level to 0]
    I -- No --> K[Keep calculated level]
    H --> L[Display daily level]
    J --> L
    K --> L
    L --> M{Threshold reached first time?}
    M -- Yes --> N[Record and display day]
    M -- No --> O{More days?}
    N --> O
    O -- Yes --> E
    O -- No --> P[Display final level]
    D --> Q([Stop])
    P --> Q
```

Task 17. Student Performance Trend Analysis

Problem Statement

Write a Python program to compare a student's marks in multiple examinations for several subjects. Show the change between the first and last examination, identify improvement or decline, and mark a subject as consistently weak if the student scores below 35 in every examination.

Algorithm

1. Start the program and read the number of subjects and examinations.

2. For each subject, create an empty list and read marks for every examination.

3. Validate each mark is between 0 and 100.

4. Calculate change = last examination mark – first examination mark.

5. If change is positive, report improvement; if negative, report decline; otherwise report no change.

6. If every mark in the subject is below 35, report the subject as consistently weak. Otherwise, report that it is not consistently weak.

7. Repeat for all subjects and display completion message.

8. Stop.

Flowchart (Text Form)

Start → Read subject and exam counts → For each subject read marks across exams → Validate marks → Compare first and last scores → Report trend → Check all scores below 35 → Report weak status → More subjects? → Stop.

Sample Input

```
Number of subjects: 2
Number of examinations: 3
Maths marks: 40, 55, 70
English marks: 20, 25, 30
```

Sample Output

```
Maths: First = 40, Last = 70, Improved by 30; Not consistently weak
English: First = 20, Last = 30, Improved by 10; Consistently weak
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Read number of subjects and exams/]
    B --> C[/Read marks for one subject/]
    C --> D{All marks between 0 and 100?}
    D -- No --> E[Report invalid marks]
    D -- Yes --> F[Change = last marks - first marks]
    F --> G{Change > 0?}
    G -- Yes --> H[Report improvement]
    G -- No --> I{Change < 0?}
    I -- Yes --> J[Report decline]
    I -- No --> K[Report no change]
    H --> L{All marks < 35?}
    J --> L
    K --> L
    L -- Yes --> M[Report consistently weak]
    L -- No --> N[Report not consistently weak]
    E --> O{More subjects?}
    M --> O
    N --> O
    O -- Yes --> C
    O -- No --> P([Stop])
```

Task 18. Fuel Consumption and Route Comparison

Problem Statement

Write a Python program to compare the fuel needs and journey costs of multiple vehicles for the same distance. Calculate journey fuel from distance and mileage, add a required reserve quantity, multiply by fuel price, and identify the lowest-cost vehicle among those with valid positive mileage.

Algorithm

1. Start the program and read the number of vehicles, journey distance, fuel price per litre, and reserve fuel quantity.
2. Validate that the distance and fuel price are positive and reserve quantity is non-negative.
3. For each vehicle, read its name and mileage.
4. If mileage is not positive, reject that vehicle.
5. Calculate journey fuel = distance / mileage.
6. Calculate total fuel = journey fuel + reserve fuel.
7. Calculate cost = total fuel × fuel price.
8. Display the vehicle's fuel requirement and cost.
9. Track the vehicle with the lowest calculated cost.
10. After all vehicles, display the lowest-cost vehicle and its cost, or report that no valid vehicle was entered.
11. Stop.

Flowchart (Text Form)

Start → Read journey details and vehicle count → Validate journey details → Read each vehicle and mileage → Valid mileage? → Calculate fuel + reserve and cost → Update lowest-cost choice → More vehicles? → Display best option → Stop.

Sample Input

```
Number of vehicles: 2
Distance: 200 km
Fuel price: ■100 per litre
Reserve fuel: 2 litres
Vehicle 1: Car, mileage 15 km/l
Vehicle 2: Bike, mileage 40 km/l
```

Sample Output

```
Car: Total fuel = 15.33 litres; Cost = ■1533.33
Bike: Total fuel = 7.00 litres; Cost = ■700.00
Lowest-cost vehicle: Bike
Lowest journey cost: ■700.00
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Read distance, fuel price, reserve, vehicle count/]
    B --> C{Journey inputs valid?}
    C -- No --> D[Display invalid input]
    C -- Yes --> E[/Read vehicle name and mileage/]
    E --> F{Mileage > 0?}
    F -- No --> G[Reject vehicle]
    F -- Yes --> H[Calculate fuel = distance / mileage]
    H --> I[Add reserve and calculate cost]
    I --> J{Cost lower than current minimum?}
    J -- Yes --> K[Update lowest-cost vehicle]
    J -- No --> L[Keep current minimum]
    G --> M{More vehicles?}
    K --> M
    L --> M
    M -- Yes --> E
    M -- No --> N[Display lowest-cost valid option]
    D --> O([Stop])
    N --> O
```

Task 19. Multi-Product Shopping Budget Constraint

Problem Statement

Write a Python program to calculate the total cost of a basket containing multiple products. Apply a 10% discount to a product line when its amount is ■1,000 or more. Compare the final basket total with the user's budget and report the amount over budget or the remaining budget. If the total is over budget, calculate a simple proportional reduction target for the basket.

Algorithm

1. Start the program and read the number of products and available budget.

2. Set basket total to zero.

3. For each product, read name, unit price, and quantity. Validate that price and quantity are non-negative.

4. Calculate line amount = price × quantity.

5. If line amount is at least ■1,000, calculate 10% discount; otherwise discount is zero.

6. Calculate line total = line amount – discount and add it to basket total.

7. After all products, compare basket total with budget.

8. If total is within budget, display remaining budget.

9. Otherwise, display the excess amount and the target basket reduction needed to meet the budget. This program reports the reduction amount; it does not choose which item to remove.

10. Stop.

Flowchart (Text Form)

Start → Read product count and budget → Read each product's price and quantity → Validate → Calculate line amount → Apply discount condition → Add line total to basket → More products? → Compare basket total with budget → Display remaining/excess amount and reduction target → Stop.

Sample Input

```
Number of products: 2
Budget: ■2000
Bag: price ■1000, quantity 1
Shoes: price ■800, quantity 1
```

Sample Output

```
Bag: Amount = ■1000; Discount = ■100; Final = ■900
Shoes: Amount = ■800; Discount = ■0; Final = ■800
Shopping Total: ■1700
Within Budget
Remaining Budget: ■300
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Read product count and budget/]
    B --> C[Set basket total = 0]
    C --> D[/Read product, price, quantity/]
    D --> E{Price and quantity valid?}
    E -- No --> F[Report invalid product input]
    E -- Yes --> G[Line amount = price x quantity]
    G --> H{Line amount >= 1000?}
    H -- Yes --> I[Discount = 10 percent]
    H -- No --> J[Discount = 0]
    I --> K[Calculate line total and add to basket]
    J --> K
    F --> L{More products?}
    K --> L
    L -- Yes --> D
    L -- No --> M{Basket total <= budget?}
    M -- Yes --> N[Display remaining budget]
    M -- No --> O[Display excess and reduction target]
    N --> P([Stop])
    O --> P
```

Task 20. University Admission Decision Engine

Problem Statement

Write a Python program to evaluate applicants for a university course. Check that each candidate meets the minimum subject marks of 35 in Mathematics, Physics, and Chemistry and an aggregate percentage of at least 60%. Display eligible applicants in descending order of percentage. If two candidates have the same percentage, use higher Mathematics marks as the tie-breaker.

Algorithm

1. Start the program and read the number of candidates.

2. For each candidate, read name and marks in Mathematics, Physics, and Chemistry.

3. Validate that every subject mark is between 0 and 100.

4. Calculate total marks and aggregate percentage = total / 3.

5. Check subject-wise eligibility: each subject must be at least 35, and aggregate percentage must be at least 60.

6. If eligible, store name, percentage, and Mathematics marks. Otherwise, report not eligible.

7. After processing all candidates, sort eligible candidates by percentage in descending order; for equal percentages, sort by Mathematics marks in descending order.

8. Display the ranked list or report that no candidates are eligible.

9. Stop.

Flowchart (Text Form)

Start → Read candidate count → Read candidate details → Validate marks → Calculate aggregate → Check subject and aggregate criteria → Store eligible candidate → More candidates? → Sort by aggregate and Maths tie-breaker → Display ranked list → Stop.

Sample Input

```
Number of candidates: 3
Ravi: Maths 90, Physics 85, Chemistry 80
Kumar: Maths 70, Physics 65, Chemistry 60
Raju: Maths 30, Physics 80, Chemistry 75
```

Sample Output

```
Ravi: Eligible, Aggregate = 85.00%
Kumar: Eligible, Aggregate = 65.00%
Raju: Not Eligible (Maths below 35)

Admission Rank List:
Rank 1: Ravi - 85.00%
Rank 2: Kumar - 65.00%
```

Mermaid Flowchart Code

```
flowchart TD
    A([Start]) --> B[/Read number of candidates/]
    B --> C[/Read candidate name and subject marks/]
    C --> D{All marks between 0 and 100?}
    D -- No --> E[Report invalid marks]
    D -- Yes --> F[Calculate aggregate percentage]
    F --> G{Each subject >= 35 and aggregate >= 60?}
    G -- Yes --> H[Store candidate and Maths marks]
    G -- No --> I[Report not eligible]
    E --> J{More candidates?}
    H --> J
    I --> J
    J -- Yes --> C
    J -- No --> K[Sort by aggregate descending; Maths marks break ties]
    K --> L[Display admission rank list]
    L --> M([Stop])
```